

What is Seaborn?

- **Seaborn** is an **amazing visualization library** for creating **statistical graphics** in Python.
 - It is **built on top of the Matplotlib library** and provides a high-level interface for attractive and informative visualizations.
 -  Seaborn is also **closely integrated with Pandas data structures**, making it easy to work with DataFrames while plotting.
-

Why use Seaborn?

- Simplifies complex visualizations.
- Provides beautiful default themes.
- Supports statistical plots easily.
- Integrates well with Matplotlib and Pandas.

Different Categories of Plots in Seaborn

Seaborn offers a variety of plotting functions grouped into different categories based on their purpose:

Relational Plots

-  Used to **understand the relationship between two variables**.
 -  Examples: `scatterplot()`, `lineplot()`, `relplot()`
-

Categorical Plots

-  Deals with **categorical variables** and visualizes how they relate to numerical data.
 -  Examples: `barplot()`, `boxplot()`, `violinplot()`, `stripplot()`, `swarmplot()`, `catplot()`
-

Distribution Plots

-  Used for examining **univariate and bivariate distributions** (spread of data).
 -  Examples: `histplot()`, `displot()`, `kdeplot()`, `ecdfplot()`, `rugplot()`
-

Regression Plots

-  Primarily intended to **add a visual guide to emphasize patterns** in data during exploratory analysis.

-  Examples: `regplot()` , `lmpplot()`
-

Matrix Plots

-  Visualizes data as a **matrix of plots or heatmaps**.
 -  Examples: `heatmap()` , `clustermap()`
-

Multi-Plot Grids

-  Useful for drawing **multiple instances of the same plot** on different subsets of a dataset.
 -  Examples: `FacetGrid` , `PairGrid` , `JointGrid` , `catplot()` , `relplot()` , `pairplot()`
-

 **Tip:** Seaborn builds on top of Matplotlib and integrates well with Pandas DataFrames, making complex visualizations easy and intuitive.

Possible Values of `palette` in Seaborn

The `palette` parameter in Seaborn controls the color scheme used for plots. It accepts a variety of predefined colormap names — both from Seaborn's native palettes and from Matplotlib's colormaps.

Available Palette Names:

Accent, Accent_r, Blues, Blues_r, BrBG, BrBG_r, BuGn, BuGn_r, BuPu, BuPu_r, CMRmap, CMRmap_r, Dark2, Dark2_r, GnBu, GnBu_r, Greens, Greens_r, Greys, Greys_r, OrRd, OrRd_r, Oranges, Oranges_r, PRGn, PRGn_r, Paired, Paired_r, Pastel1, Pastel1_r, Pastel2, Pastel2_r, PiYG, PiYG_r, PuBu, PuBuGn, PuBuGn_r, PuBu_r, PuOr, PuOr_r, PuRd, PuRd_r, Purples, Purples_r, RdBu, RdBu_r, RdGy, RdGy_r, RdPu, RdPu_r, RdYIBu, RdYIBu_r, RdYIGn, RdYIGn_r, Reds, Reds_r, Set1, Set1_r, Set2, Set2_r, Set3, Set3_r, Spectral, Spectral_r, Wistia, Wistia_r, YlGn, YlGnBu, YlGnBu_r, YlGn_r, YlOrBr, YlOrBr_r, YlOrRd, YlOrRd_r, afmhot, afmhot_r, autumn, autumn_r, binary, binary_r, bone, bone_r, brg, brg_r, bwr, bwr_r, cividis, cividis_r, cool, cool_r, coolwarm, coolwarm_r, copper, copper_r, cubehelix, cubehelix_r, flag, flag_r, gist_earth, gist_earth_r, gist_gray, gist_gray_r, gist_heat, gist_heat_r, gist_ncar, gist_ncar_r, gist_rainbow, gist_rainbow_r, gist_stern

```
In [1]: import seaborn as sns
```

```
In [2]: df=sns.load_dataset('tips')
```

```
In [120... df
```

Out[120...

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4
...	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3
240	27.18	2.00	Female	Yes	Sat	Dinner	2
241	22.67	2.00	Male	Yes	Sat	Dinner	2
242	17.82	1.75	Male	No	Sat	Dinner	2
243	18.78	3.00	Female	No	Thur	Dinner	2

244 rows × 7 columns

In []: `df.head()`

Out[]:

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

HISTPLOT



What is `histplot()` in Seaborn?

The `histplot()` function in Seaborn is used to **visualize the distribution of a single variable or the relationship between two variables** as a histogram.

It can display **counts or probability densities** and supports optional kernel density estimates (KDE) as well.



Why Use `histplot()`?

- Shows how values are distributed over intervals.
- Can display **univariate or bivariate histograms**.
- Supports additional features like **hue-based separation, multiple bins**, and **density estimates**.
- Allows easy customization of bins, colors, KDE overlays, and more.

Syntax

```
seaborn.histplot(  
    data=None, *, x=None, y=None, hue=None, weights=None,  
    stat='count', bins='auto', binwidth=None, discrete=None,  
    cumulative=False, common_bins=True, common_norm=True,  
    multiple='layer', element='bars', fill=True, kde=False,  
    color=None, palette=None, legend=True, **kwargs  
)
```

📌 Quick Tip:

 Use `multiple='stack'` to stack histograms.

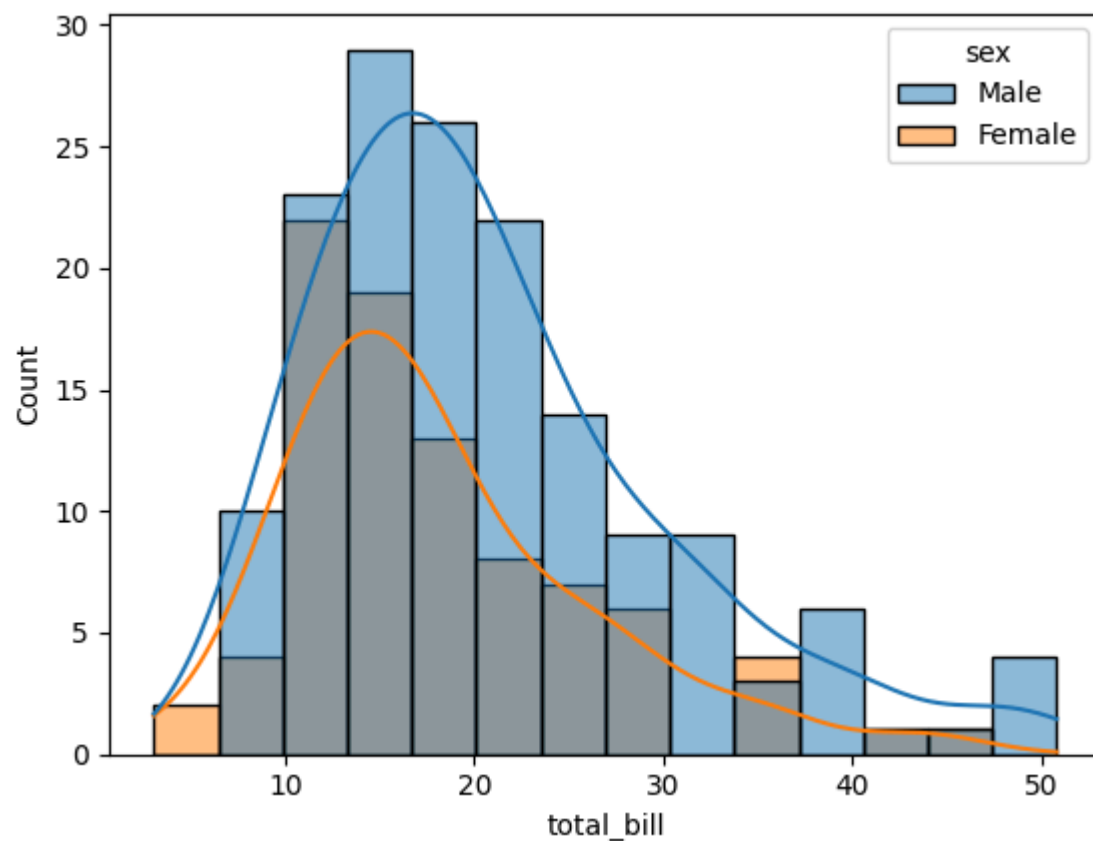
 Set `kde=True` to overlay a kernel density curve.

 Use `hue` for comparing distributions by category.

 Parameter	 Description
data	Dataset (usually Pandas DataFrame).
x , y	Variables to be plotted on the axes.
hue	Categorical variable that separates data by color.
stat	Aggregation to compute: "count" (default), "frequency" , "probability" , or "density" .
bins	Number of histogram bins or binning strategy (default "auto").
binwidth	Width of each bin (overrides bins).
discrete	Set True if the variable is discrete.
cumulative	If True , plot cumulative counts.
multiple	How to handle multiple observations at the same value: "layer" , "dodge" , "stack" , or "fill" .
element	Visual representation: "bars" (default), "step" , or "poly" (polygon).
fill	If True , fill bars; else draw outlines only.
kde	If True , overlay a KDE curve.
color	Color for all elements.
palette	Color palette for different hue levels.
legend	Show or hide the legend.
`kwargs`**	Additional keyword arguments for Matplotlib's <code>bar()</code> function.

```
In [4]: sns.histplot(data=df, x='total_bill', hue='sex', color='m', kde=True )
```

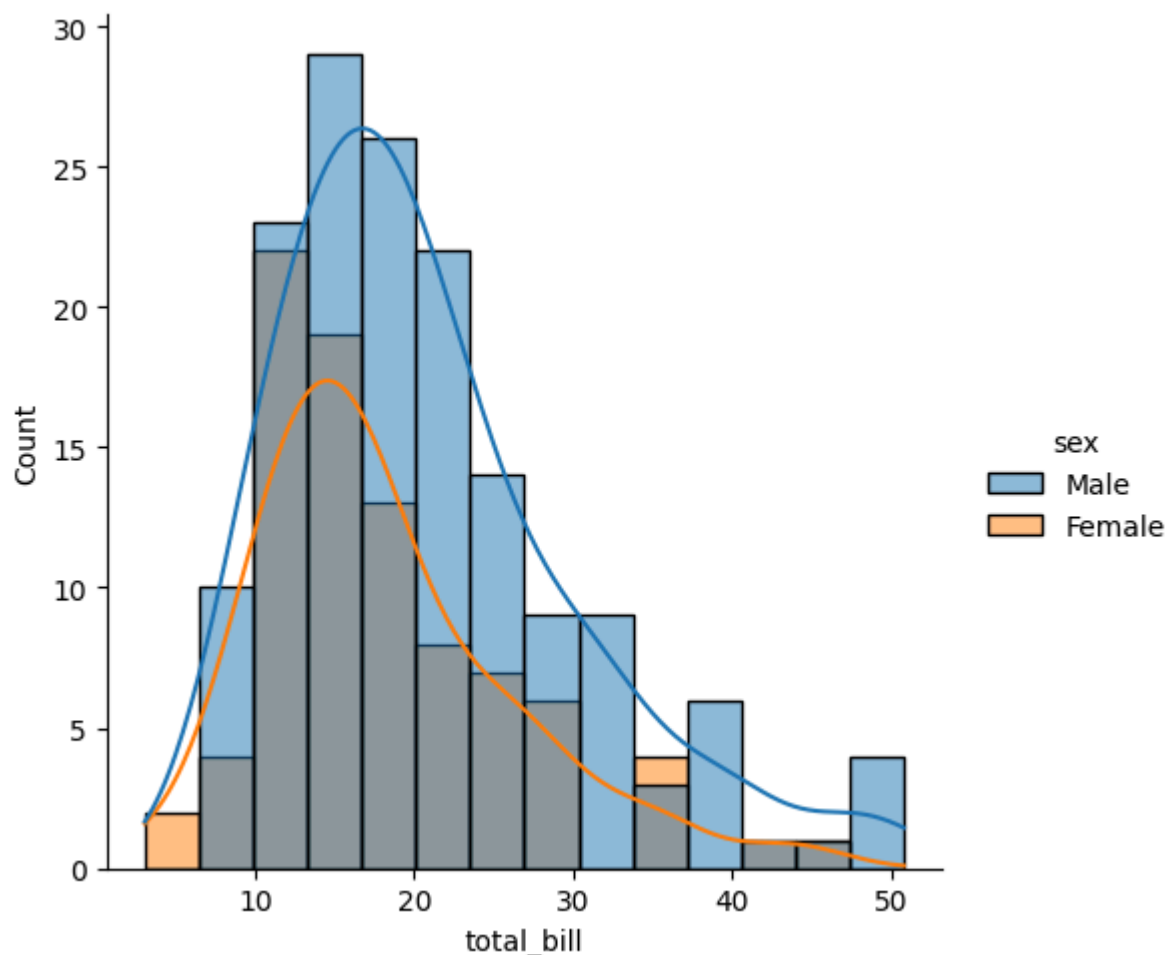
```
Out[4]: <Axes: xlabel='total_bill', ylabel='Count'>
```



Distplot = deprecated in newer version
now can call it as 'displot'.

```
In [5]: sns.displot(data=df,x='total_bill',hue='sex',color='m',kde=True )
```

```
Out[5]: <seaborn.axisgrid.FacetGrid at 0x1fedde20320>
```



Difference Between `displot()` and `histplot()` in Seaborn

 Feature	 <code>displot()</code>	 <code>histplot()</code>
 Type	Figure-level function	Axes-level function
 Returns	Creates an entire Figure (can have multiple subplots)	Adds a single plot to an existing Axes
 When to Use	When you want to create standalone, full-figure plots (often with <code>FacetGrid</code> for multiple plots in one figure)	When you want to draw a histogram inside an existing subplot or axes
 Faceting (multiple subplots)	✅ Supported (with <code>col</code> , <code>row</code> , <code>hue</code>)	❌ Not supported
 Control Level	Less fine control (since it manages figure layout for you)	More control over axis and subplot management
 Syntax Example	<pre>sns.displot(data, x="column")</pre>	<pre>sns.histplot(data, x="column")</pre>
 KDE Support	✅ Can add kernel density estimate (<code>kde=True</code>)	✅ Can add KDE as well (<code>kde=True</code>)
 Combine Multiple Histograms	✅ Can easily combine and facet by a categorical variable (<code>hue</code>)	✅ Can combine but harder to facet
 Use Case	Quick full-figure distribution plots, optionally faceted by one or more variables	Customizing individual histograms inside a figure or subplot

Quick Summary:

- Use `displot()` when you need a **complete figure with multiple plots or automatic faceting**.
- Use `histplot()` when you're plotting **a single histogram or integrating into an existing figure layout**.

Heatmap

Syntax and Parameters of `seaborn.heatmap()`

Syntax

`seaborn.heatmap(data, *, vmin=None, vmax=None, cmap=None, center=None, annot=None, annot_kws=None, linewidths=0, linecolor='white', cbar=True, **kwargs)`

 Parameter	 Description
<code>data</code>	 A 2D dataset that can be converted into a NumPy <code>ndarray</code> . (like a DataFrame or list of lists)
<code>vmin</code> , <code>vmax</code>	 Values to anchor the colormap . If not given, they are inferred from the data's minimum and maximum values.
<code>cmap</code>	 The colormap for mapping data values to colors. Can be a list of colors or predefined color map names like <code>'coolwarm'</code> , <code>'tab20'</code> , etc.
<code>center</code>	 Value at which to center the colormap when dealing with divergent data.
<code>annot</code>	 If <code>True</code> , it displays numerical values inside the cells .
<code>fmt</code>	 String format for annotations (like <code>"d"</code> for integer, <code>".2f"</code> for float).
<code>linewidths</code>	 Width of the lines separating the heatmap cells.
<code>linecolor</code>	 Color of the separating lines between cells.
<code>cbar</code>	 Whether to display a color bar alongside the heatmap.
<code>annot_kws</code>	 Dictionary of keyword arguments for customizing the annotation text (like font size, color).

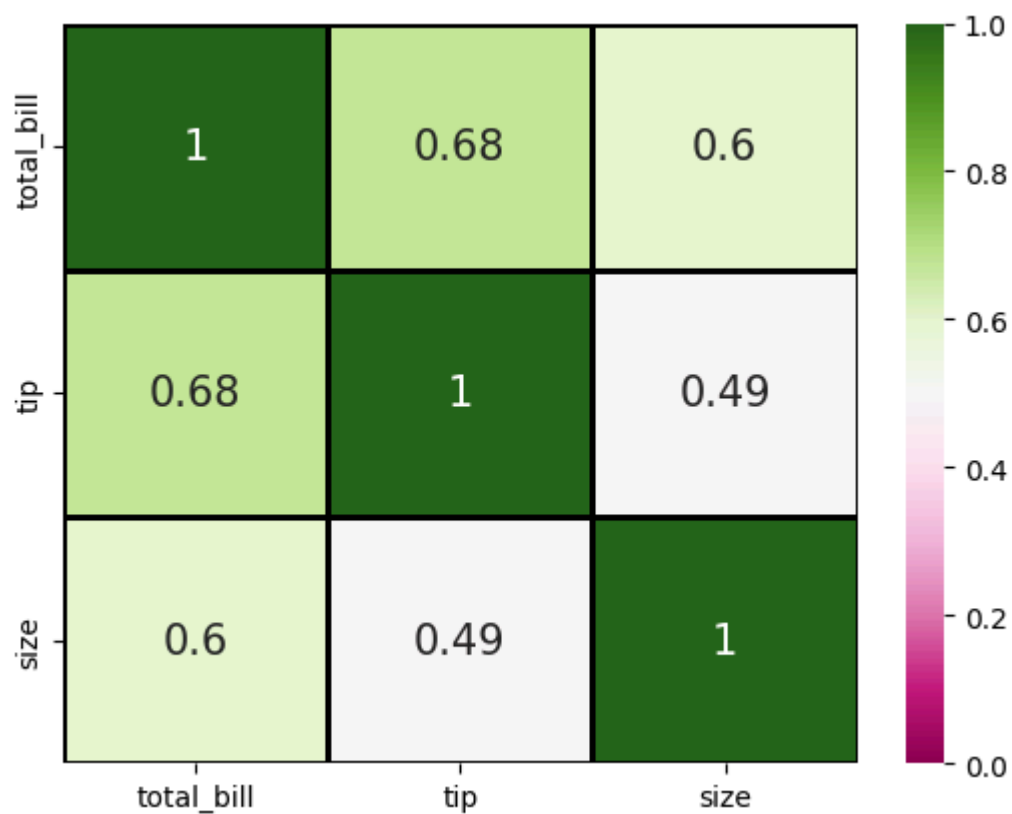
a

```
In [6]: data=df.corr(numeric_only=True)
```

```
In [7]: corelation=df.select_dtypes(include=['int','float']).corr()
```

```
In [8]: sns.heatmap(data, vmin=0,vmax=1,center=0.5, cmap='PiYG',annot=True,lw=2,linecolor='k',annot_k
```

```
Out[8]: <Axes: >
```

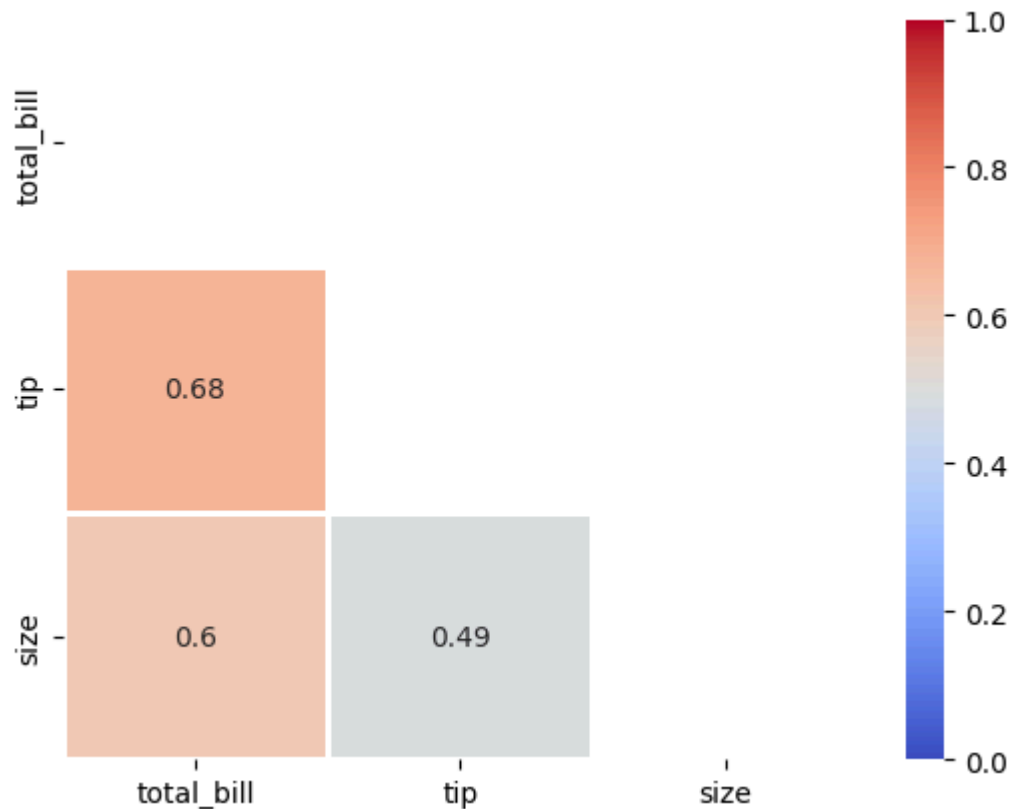


Triangle correlation heatmap

```
In [9]: import numpy as np
mask = np.triu(np.ones_like(data.corr()))
```

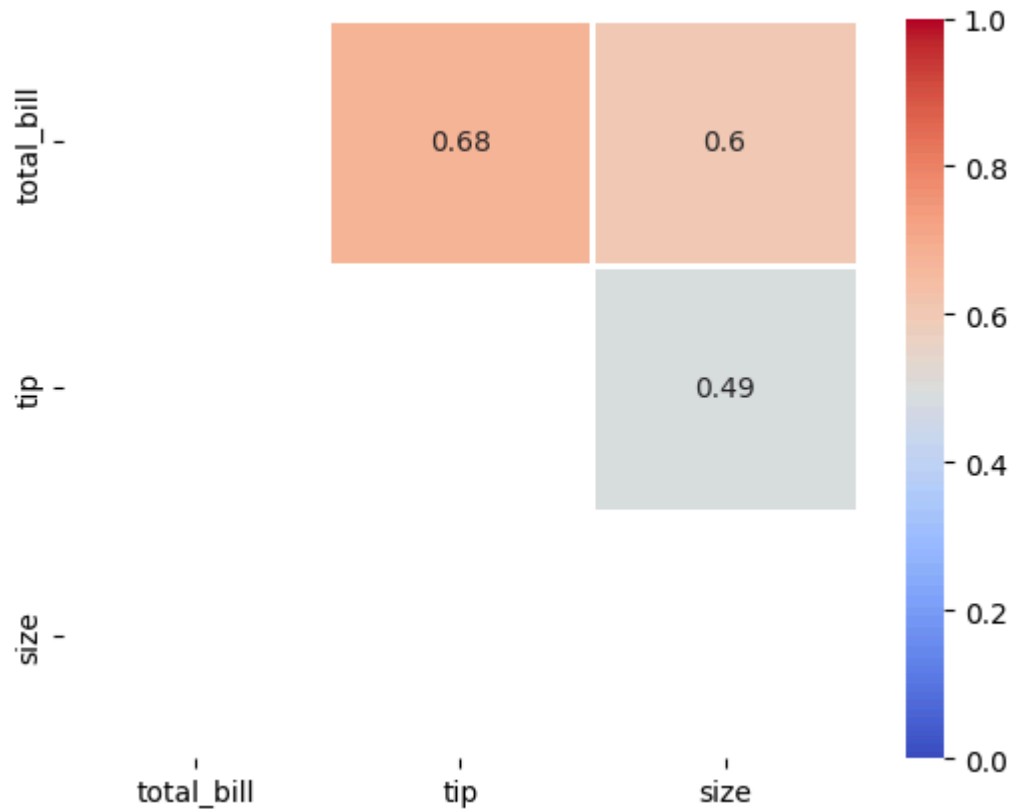
```
In [10]: sns.heatmap(data,mask=mask,cmap='coolwarm',vmin=0,vmax=1,center=0.5,lw=2,linewidth='white',an
```

Out[10]: <Axes: >



```
In [11]: mask = np.tril(np.ones_like(data.corr()))
sns.heatmap(data,mask=mask,cmap='coolwarm',vmin=0,vmax=1,center=0.5,lw=2,linewidth='white',an
```

Out[11]: <Axes: >



Jointplot

What is `jointplot()` in Seaborn?

The `jointplot()` function in Seaborn is a **figure-level function** used to **visualize the relationship between two variables along with their univariate distributions**.

It creates a **bivariate plot (scatter, KDE, or regression)** with **marginal histograms or KDE plots** on the top and right sides.

Why Use `jointplot()` ?

-  Shows the **relationship between two variables**.
-  Displays the **distribution of each variable individually** alongside.
-  Supports different kinds of plots: `"scatter"`, `"kde"`, `"hist"`, `"hex"`, and `"reg"`.

Syntax

```
seaborn.jointplot(  
    x=None, y=None, data=None, kind='scatter',  
    color=None, height=6, ratio=5, space=0.2,  
    dropna=True, xlim=None, ylim=None, **kwargs  
)
```

 Quick Tip:
`"kde"` kind draws density estimate plots.

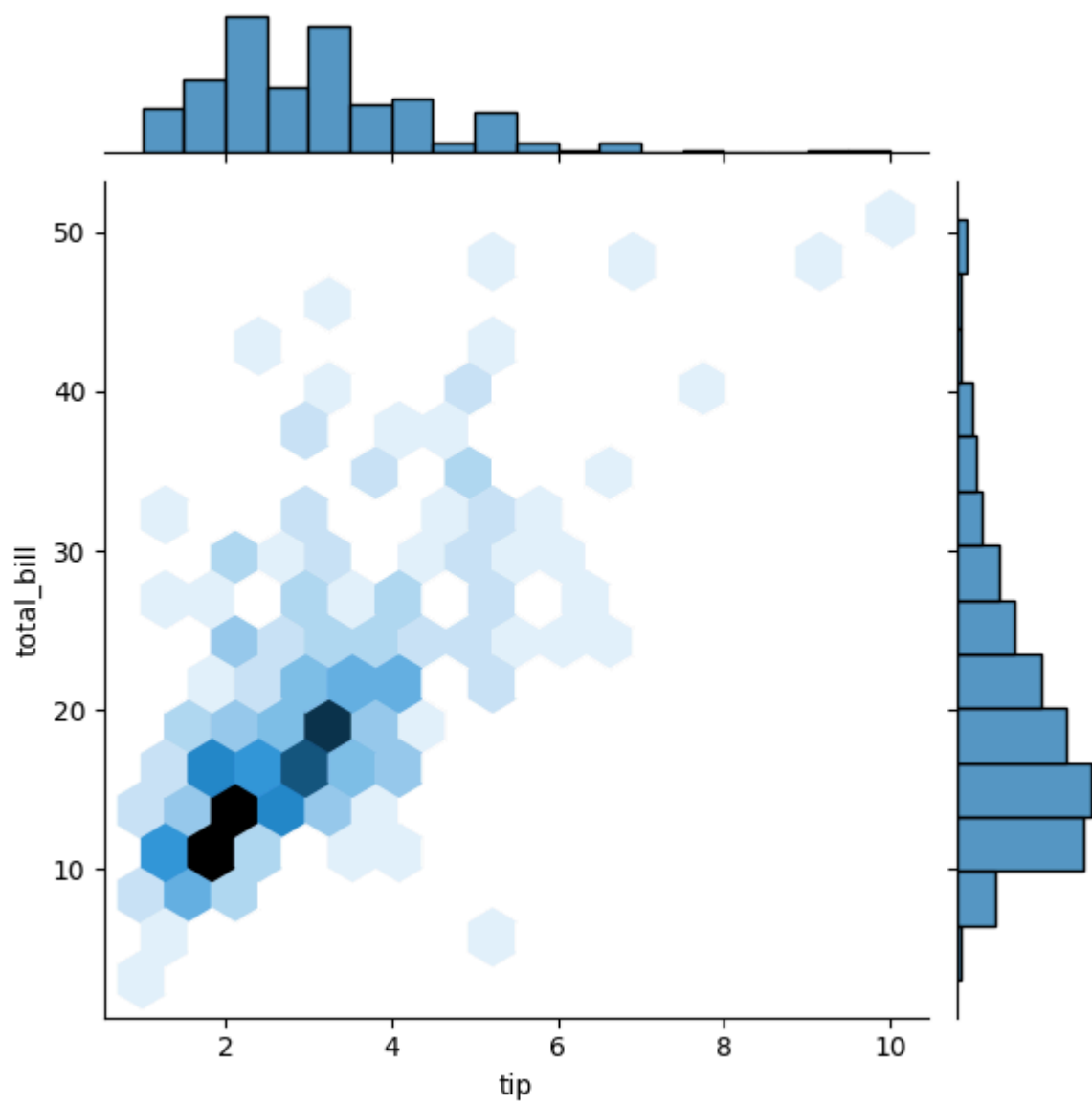
"hex" kind draws a hexbin plot **for** dense data.

"reg" kind fits a regression line along **with** scatter.

 Parameter	 Description
x , y	Names of variables in <code>data</code> .
data	Pandas DataFrame containing the variables.
kind	Type of plot to draw: "scatter" (default), "kde" , "hist" , "hex" , or "reg" .
color	Color for all elements.
height	Size of the figure (default is <code>6</code>).
ratio	Ratio of joint axes height to marginal axes height (default is <code>5</code>).
space	Space between joint and marginal axes (default is <code>0.2</code>).
dropna	If <code>True</code> , removes missing values before plotting.
xlim , ylim	Limits for X and Y axes.
`kwargs`**	Additional keyword arguments for specific kind of plot.

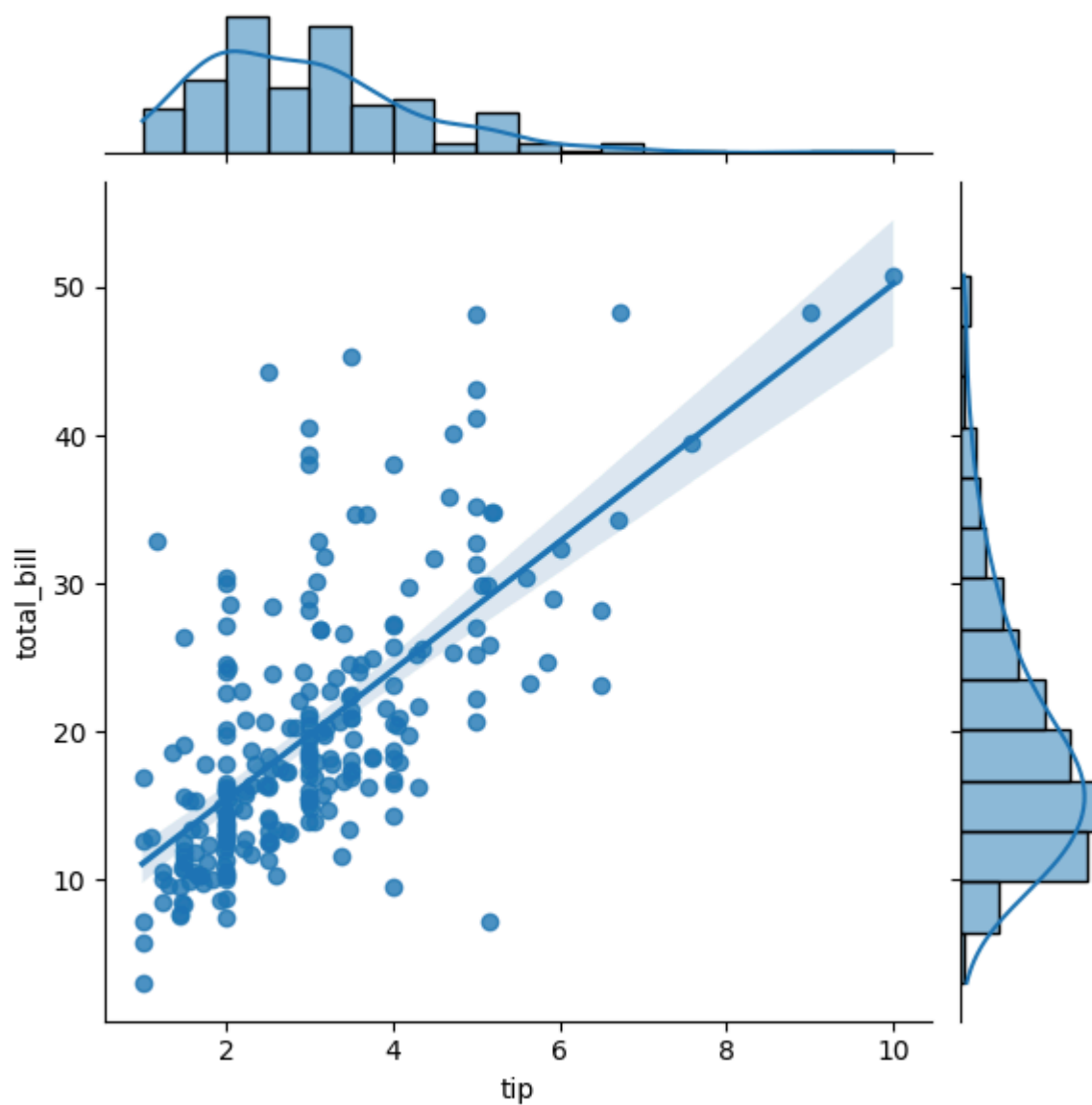
```
In [12]: sns.jointplot(data=df,x='tip',y='total_bill',kind="hex")
```

```
Out[12]: <seaborn.axisgrid.JointGrid at 0x1fee2656480>
```



```
In [13]: sns.jointplot(data=df,x='tip',y='total_bill',kind="reg")
```

```
Out[13]: <seaborn.axisgrid.JointGrid at 0x1feddef6060>
```



Pairplot

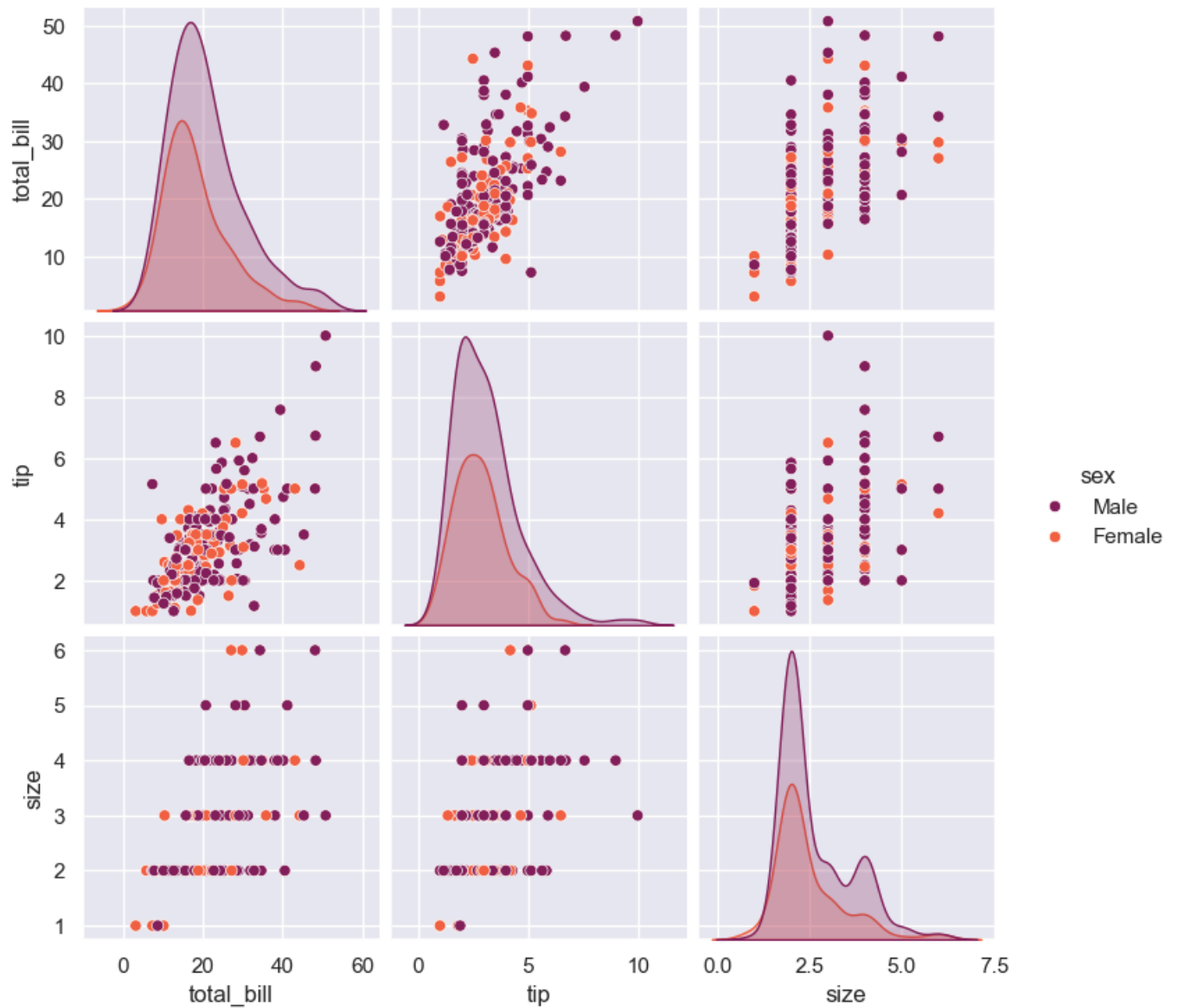
Pairplot Argument Descriptions

 Argument	 Description	 Value
data	DataFrame where each column is a variable and each row is an observation.	DataFrame
hue	Variable in <code>data</code> to map plot aspects to different colors.	string (variable name), <i>optional</i>
palette	Set of colors for mapping the <code>hue</code> variable. Can be a dictionary for custom color mapping.	dict or seaborn color palette
{x, y}_vars	Allows you to specify which variables to use for the rows and columns of the plot to create a custom layout.	Lists of variable names, <i>optional</i>
dropna	Drop missing values from the data before plotting.	boolean, <i>optional</i>

 **Tip:** This is typically used in `sns.pairplot()` to explore relationships between numerical variables and optionally color by a categorical feature.

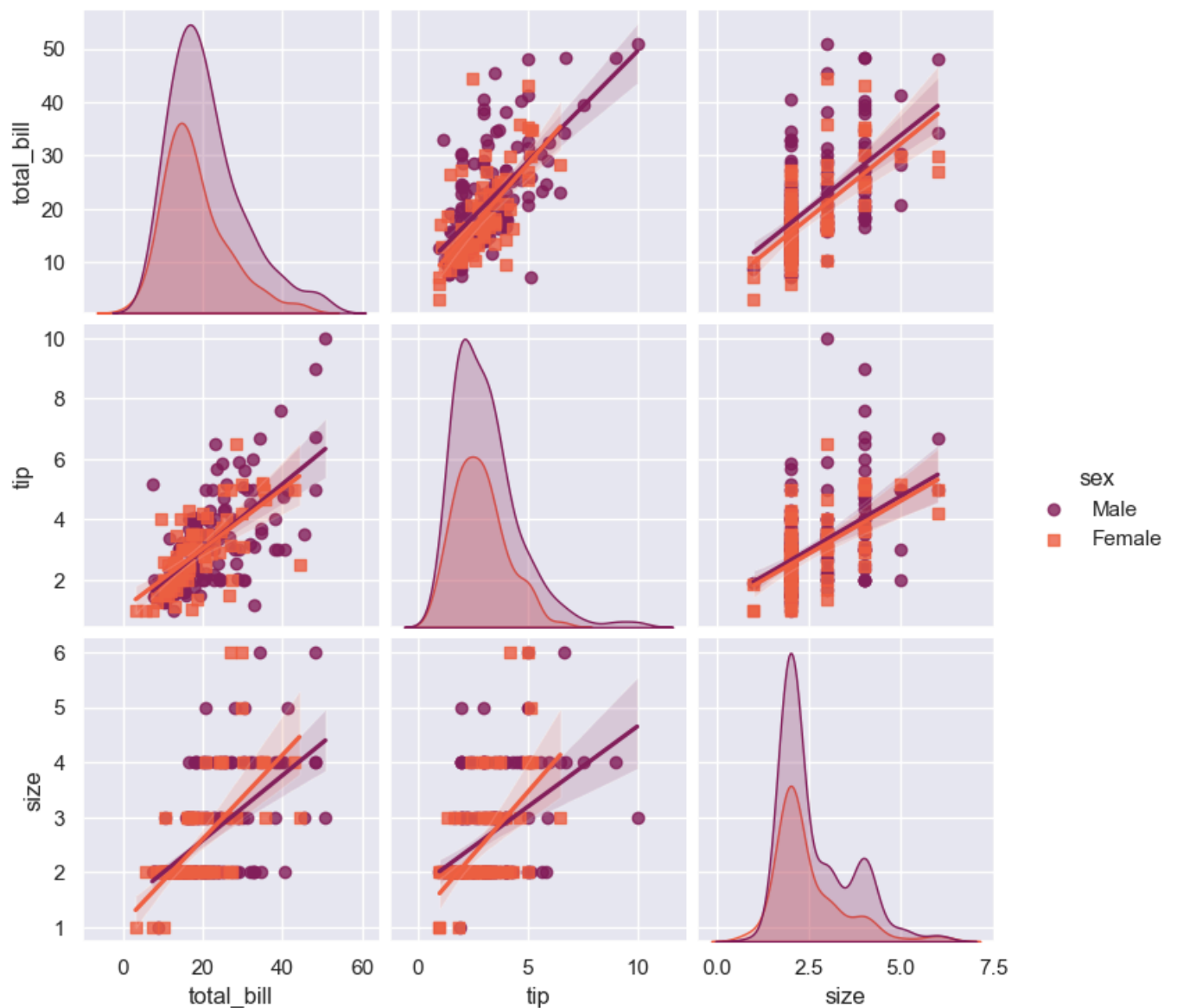
```
In [14]: sns.set_style('darkgrid')
sns.set_context('notebook')
sns.pairplot(df,hue='sex',palette='rocket',dropna=True)
```

Out[14]: <seaborn.axisgrid.PairGrid at 0x1fee2657710>



```
In [15]: sns.pairplot(df,hue='sex',palette='rocket',dropna=True,kind='reg',markers=['o','s'])
```

Out[15]: <seaborn.axisgrid.PairGrid at 0x1fee4425ac0>



Countplot



Syntax and Parameters of `seaborn.countplot()`



Syntax

```
seaborn.countplot(
    x=None, y=None, hue=None, data=None, order=None, hue_order=None, orient=None,
    color=None, palette=None, saturation=0.75, dodge=True, ax=None, **kwargs
)
```



Parameter

Description

`x`, `y`

Names of variables in `data` or vector data, optional. Inputs for plotting long-form data.

`hue`

(optional) Column name for color encoding.

`data`

(optional) DataFrame, array, or list of arrays. Dataset for plotting. If `x` and `y` are absent, this is interpreted as wide-form; otherwise, long-form.

 Parameter	 Description
order , hue_order	(optional) Lists of strings. Order to plot the categorical levels in. If not provided, levels are inferred from the data.
orient	(optional) "v" "h". Orientation of the plot (vertical or horizontal). Usually inferred but can be manually set.
color	(optional) Matplotlib color. Color for all the elements, or seed for a gradient palette.
palette	(optional) Palette name, list, or dict. Colors to use for different levels of the <code>hue</code> variable. Can be a palette name or a dictionary mapping hue levels to colors.
saturation	(optional) Float. Proportion of the original saturation for colors. Use lower values for large patches for better appearance. Default is <code>0.75</code> .
dodge	(optional) Boolean. When hue nesting is used, determines whether elements should be shifted along the categorical axis.
ax	(optional) Matplotlib Axes. Axes object to draw the plot onto. If not provided, uses the current Axes.
<code>`kwargs`</code>	Additional keyword arguments passed to <code>matplotlib.axes.Axes.bar()</code> .

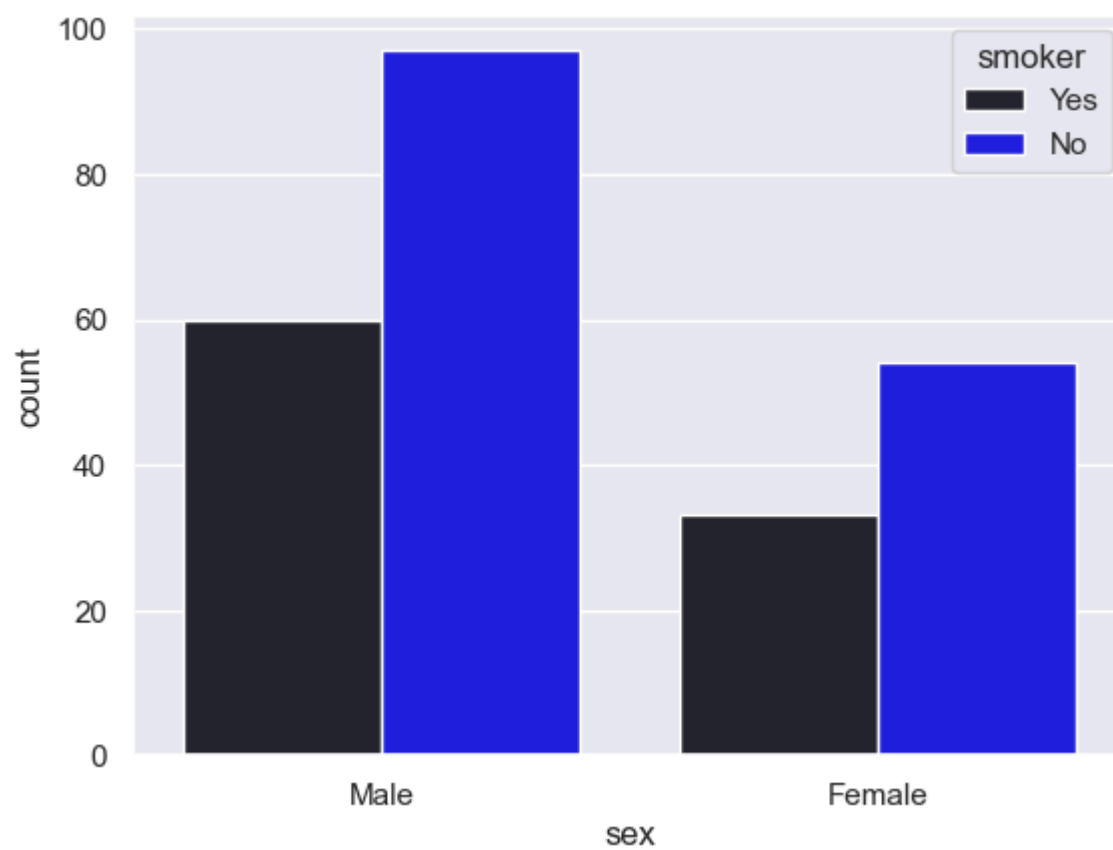
```
In [17]: sns.countplot(x='sex',hue='smoker',data=df,color='b',)
```

C:\Users\ajink\AppData\Local\Temp\ipykernel_9060\3678068160.py:1: FutureWarning:

Setting a gradient palette using color= is deprecated and will be removed in v0.14.0. Set `palette='dark:b'`` for the same effect.

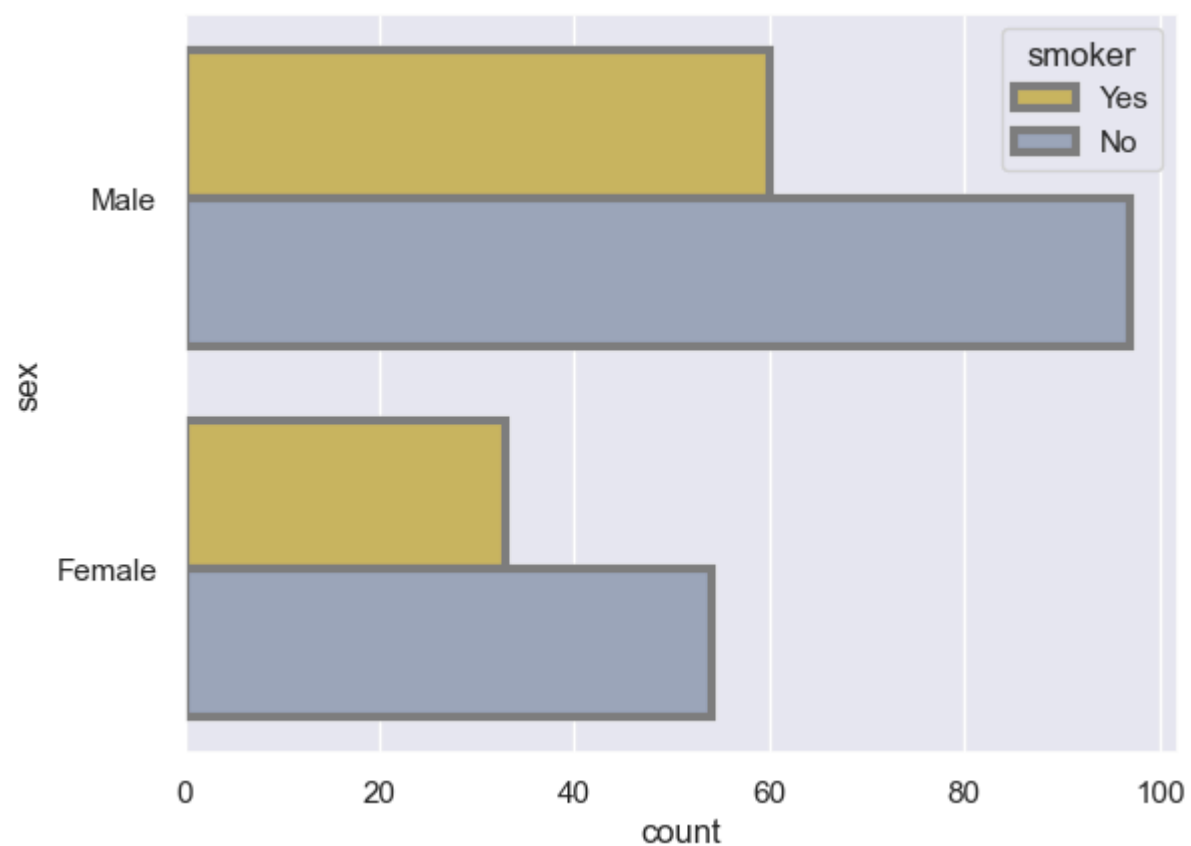
```
sns.countplot(x='sex',hue='smoker',data=df,color='b',)
```

```
Out[17]: <Axes: xlabel='sex', ylabel='count'>
```



In [36]: `sns.countplot(y='sex',hue='smoker',data=df,palette='Set2_r',saturation=0.5,lw=3,edgecolor='gr`

Out[36]: `<Axes: xlabel='count', ylabel='sex'>`



Boxplot



What is a Boxplot?

A **Boxplot** is a statistical graph used to **visualize the distribution of numerical data** through their quartiles and to identify key summary statistics.

📌 Key Uses of a Boxplot:

- 📊 Shows the **distribution of numerical data**.
 - 📏 Identifies key statistics:
 - **Minimum value**
 - **Maximum value**
 - **Median (Q2)**
 - **First Quartile (Q1)**
 - **Third Quartile (Q3)**
 - 🔍 Helps in **identifying outliers**.
 - 📊 Useful for **understanding how data is distributed** (skewed, symmetric, etc.).
 - 📄 Can **compare the distribution of data across different categories or variables** by using the `hue` or `x` parameter in Seaborn.
-

📊 Syntax and Parameters of `seaborn.boxplot()`

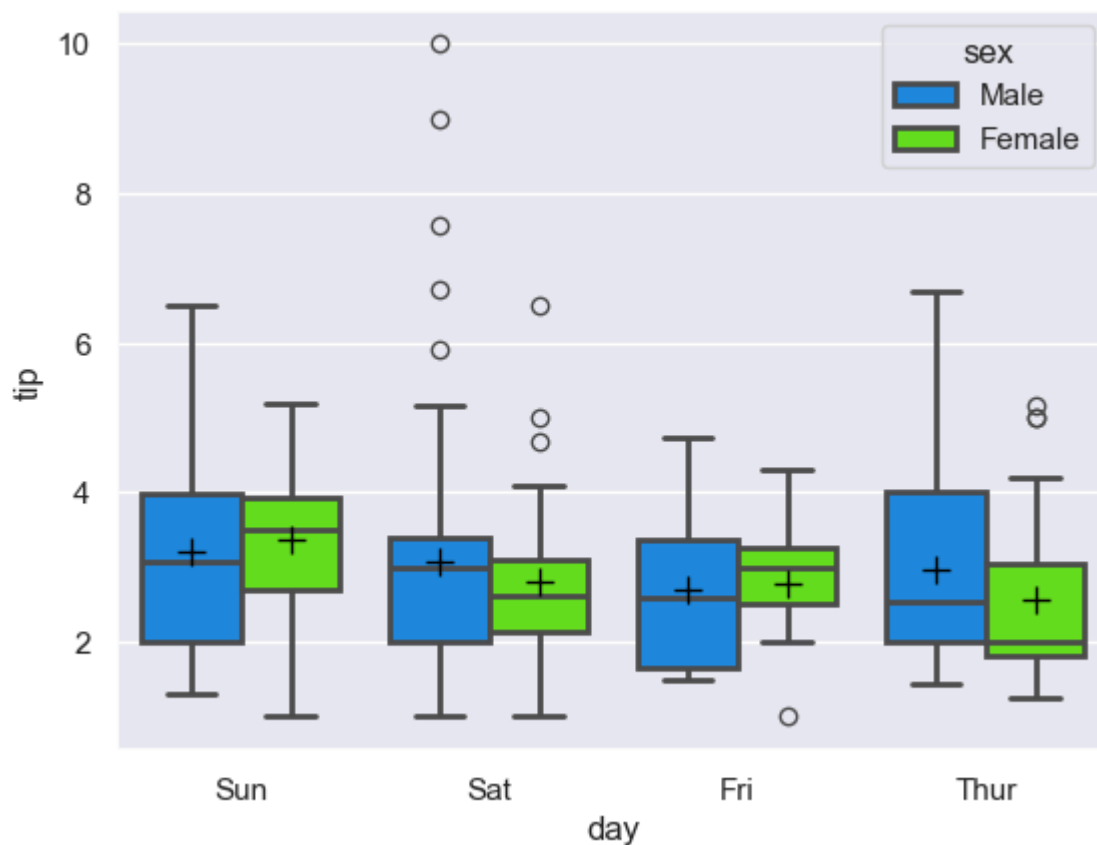
📌 Syntax

```
seaborn.boxplot(  
    x=None, y=None, hue=None, data=None, color=None, palette=None, linewidth=None,  
    **kwargs  
)
```

📄 Parameter	📄 Description
<code>x, y, hue</code>	Inputs for plotting long-form data (column names from <code>data</code> or vector data).
<code>data</code>	Dataset for plotting. If <code>x</code> and <code>y</code> are absent, this is interpreted as wide-form data (multiple columns treated as separate distributions).
<code>color</code>	Color for all of the elements in the boxplot.
<code>palette</code>	(optional) Colors to use for the different levels of the <code>hue</code> variable.
<code>linewidth</code>	(optional) Width of the lines that form the boxplot elements.
<code>`kwargs`**</code>	Additional keyword arguments passed through to <code>matplotlib</code> 's <code>Axes.boxplot()</code> function.

```
In [44]: sns.boxplot(x='day',y='tip',data=df,hue='sex',orient='v',order=['Sun','Sat','Fri','Thur'],pal  
            "markedgedcolor": "black", "markersize": "10"))
```

```
Out[44]: <Axes: xlabel='day', ylabel='tip'>
```



Violin plot



What is a Violin Plot?

A **Violin Plot** is a combination of a **boxplot** and a **kernel density plot** that depicts the distribution of numeric data for one or more groups using **density curves**.



Key Uses of a Violin Plot:

- Visualizes the **distribution of numeric data**.
- The **width of each curve** represents the **approximate frequency of data points** in each region.
- Can overlay other chart types (like a **boxplot**) inside the violin to provide additional summary statistics.
- Useful for **comparing distributions across different categories**.



Syntax

```
seaborn.violinplot(
    x=None, y=None, hue=None, data=None, order=None, hue_order=None, bw='scott',
    cut=2,
    scale='area', scale_hue=True, gridsize=100, width=0.8, inner='box', split=False,
    dodge=True, orient=None, linewidth=None, color=None, palette=None,
    saturation=0.75, ax=None, **kwargs
)
```

 Parameter	 Description
<code>x</code> , <code>y</code> , <code>hue</code>	Inputs for plotting long-form data (column names from <code>data</code>).
<code>data</code>	Dataset for plotting.
<code>scale</code>	Method used to scale the width of each violin. Options: <code>'area'</code> , <code>'count'</code> , <code>'width'</code> .
<code>bw</code>	(optional) The bandwidth of the kernel; affects the smoothness of the density curve.
<code>cut</code>	(optional) Defines how far the density curve extends past extreme data points.
<code>inner</code>	(optional) Type of plot to draw inside the violin: <code>'box'</code> , <code>'quartile'</code> , <code>'point'</code> , or <code>'stick'</code> .
<code>palette</code>	(optional) Colors to use for the different levels of the <code>hue</code> variable.
<code>linewidth</code>	(optional) Width of the lines outlining the violin shapes.
<code>color</code>	(optional) Color for all violins.
<code>ax</code>	(optional) Axes object to draw the plot onto.
<code>`kwargs`**</code>	Additional keyword arguments passed to <code>matplotlib</code> .

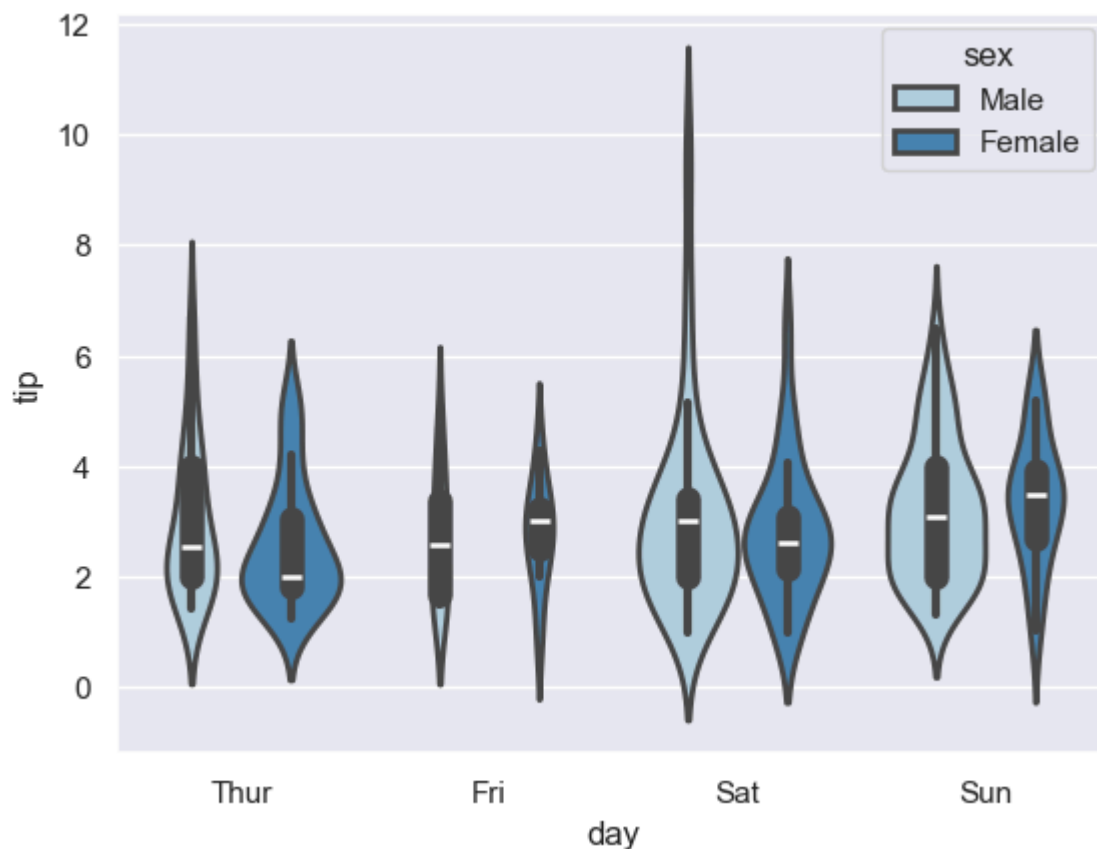
```
In [54]: sns.violinplot(x='day',y='tip',data=df,hue='sex',palette='Blues',linewidth=2,scale='count')
```

C:\Users\ajink\AppData\Local\Temp\ipykernel_9060\307205085.py:1: FutureWarning:

The ``scale`` parameter has been renamed and will be removed in v0.15.0. Pass ``density_norm='count'`` for the same effect.

```
sns.violinplot(x='day',y='tip',data=df,hue='sex',palette='Blues',linewidth=2,scale='count')
```

```
Out[54]: <Axes: xlabel='day', ylabel='tip'>
```



Seaborn FacetGrid vs PairGrid — Quick Interview Summary

FacetGrid

-  Used to **draw the same plot on multiple subsets of a dataset**.
-  Creates a grid of subplots based on the **values of one or more categorical variables**.
-  Good for **visualizing how a distribution or relationship varies across categories**.
-  Can map any Seaborn or Matplotlib plotting function onto each facet.

Example

```
import seaborn as sns
import matplotlib.pyplot as plt

# Load example dataset
tips = sns.load_dataset("tips")

# Create FacetGrid and map a histogram
g = sns.FacetGrid(tips, col="sex")
g.map(sns.histplot, "total_bill")
plt.show()
```

Seaborn PairGrid

What is PairGrid?

-  PairGrid is a **multi-axes grid** for plotting **pairwise relationships** in a dataset.
-  It allows you to **map different Seaborn plotting functions** to the **upper, lower, and diagonal** sections of a grid.
-  More **customizable than pairplot()**, as you can define exactly what kind of plot should appear in each section.
-  Useful for **exploring interactions between multiple numeric variables** in a dataset.

Important Methods

- `.map_diag()` → for diagonal plots
- `.map_upper()` → for upper triangle plots
- `.map_lower()` → for lower triangle plots

Example

```
import seaborn as sns
import matplotlib.pyplot as plt

# Load example dataset
iris = sns.load_dataset("iris")
```

```
# Create a PairGrid
g = sns.PairGrid(iris)

# Map plots to different sections
g.map_diag(sns.histplot)
g.map_upper(sns.scatterplot)
g.map_lower(sns.kdeplot)

plt.show()
```

📌 When to Use

When you need full control over the pairwise plot grid layout.
 When you want to use different plot types **for** different parts of the grid.
 When pairplot() feels too limited **for** custom visualizations.

🎯 Bonus Tip:

- 📊 Use pairplot() **for** quick pairwise plotting.
- 📊 Use PairGrid when you want different kinds of plots **in** the upper, lower, **and** diagonal grids **or** need extra customization.

📊 FacetGrid

Draws the **same plot** for subsets of data across multiple subplots.

Good for **faceting** (splitting data into subplots by category).

📊 PairGrid

Draws **different pairwise plots** (scatter, hist, etc.) for multiple variable combinations.

Good for **pairwise exploration** of multiple numerical variables.

Scatterplot

📊 What is a Scatterplot?

A **scatter plot** displays points on a **two-dimensional axis** to show the **relationship between two variables**.

Each dot represents an observation from your dataset.

📌 What You Can Do with Seaborn's scatterplot()

- 📊 Show **relationships between two numeric variables**.
- 🎨 Differentiate groups using:
 - **hue** → color
 - **style** → marker shape
 - **size** → point radius
- 🗺️ Improve **accessibility and clarity with semantic mapping**.
- 📄 Add legends, adjust transparency, and customize aesthetics.

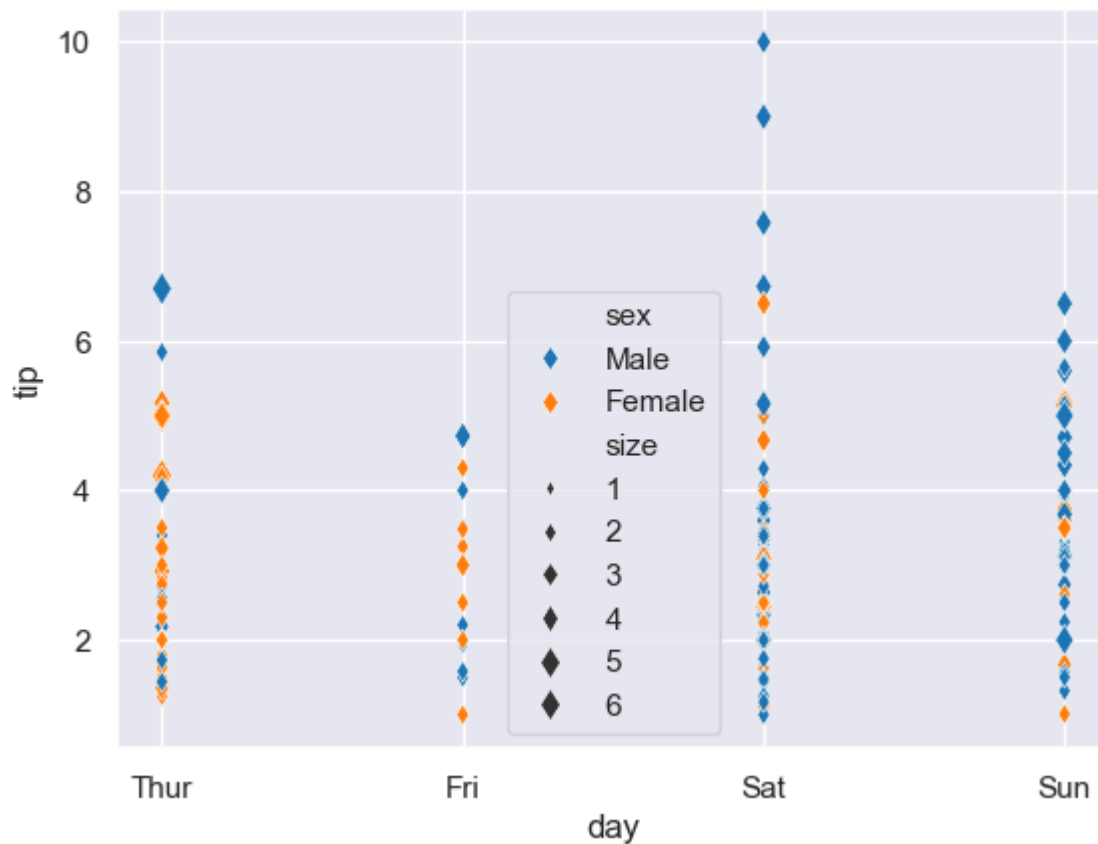
📌 Syntax

```
seaborn.scatterplot(
    x=None, y=None,
    hue=None, style=None, size=None,
    data=None, palette=None,
    legend='brief', alpha='auto',
    **kwargs
)
```

 Parameter	 Description
x, y	Numeric variables for the X and Y axes.
data	Pandas DataFrame.
hue	Variable that maps to different colors.
style	Variable that maps to different marker shapes.
size	Variable that maps to different marker sizes.
palette	Defines the color palette.
alpha	Transparency of the points (0.0 to 1.0).
legend	Controls legend visibility and behavior.

```
In [63]: sns.scatterplot(x='day',y='tip',data=df,color='r',size='size',hue='sex',marker='d')
```

```
Out[63]: <Axes: xlabel='day', ylabel='tip'>
```



Lineplot



What is a Line Plot?

A **line plot** presents data on an **X-Y axis using a continuous line** that connects data points. It's useful for **visualizing trends over a continuous interval** like time, categories, or sequences.

Seaborn `lineplot()` Function

Seaborn provides an inbuilt function called `lineplot()` to easily create line plots.

Syntax

```
seaborn.lineplot(x, y, data)
```

Error Bars in Line Plot

Error bars are visual indicators that show **error margins or variability** in a line plot, helping to study confidence intervals or data spread at each point.

How to Add Error Bars in Seaborn's `lineplot()`

Seaborn's `lineplot()` function supports error bars using the `err_style` attribute.

`err_style` Options

- `"band"` → Shaded confidence interval band (default)
 - `"bars"` → Vertical error bars at each data point
-

Example:

```
import seaborn as sns
import matplotlib.pyplot as plt

# Load example dataset
fmri = sns.load_dataset("fmri")

# Lineplot with error bars as bands
sns.lineplot(x="timepoint", y="signal", data=fmri, err_style="band")

# Lineplot with error bars as bars
sns.lineplot(x="timepoint", y="signal", data=fmri, err_style="bars")

plt.show()
```

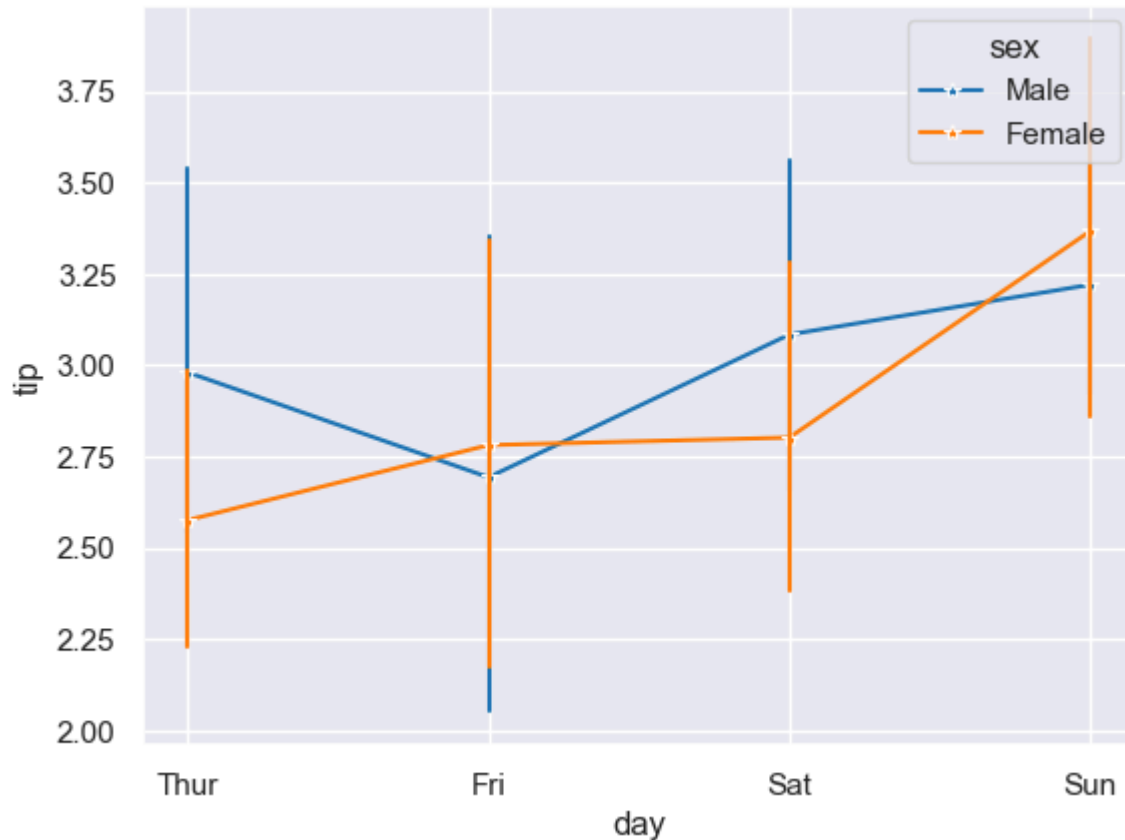
Note

You can also control the confidence interval range using the `ci` parameter (e.g., `ci=95` for a 95% interval).

By default, Seaborn uses `ci=95` and `err_style='band'`.

```
In [80]: sns.lineplot(x='day',y='tip',data=df,hue='sex',marker='*',err_style="bars")
```

```
Out[80]: <Axes: xlabel='day', ylabel='tip'>
```



Barplot



What is a Bar Plot?

A **bar plot** represents an **estimate of central tendency (like mean or median)** for a numeric variable with the **height of each rectangle (bar)**.

It also provides some **indication of uncertainty** around that estimate using **error bars**.



Seaborn `barplot()` Function



Syntax

```
seaborn.barplot(  
    x=None, y=None, hue=None, data=None, order=None, hue_order=None,  
    estimator=<function mean>, ci=95, n_boot=1000, units=None, seed=None,  
    orient=None, color=None, palette=None, saturation=0.75,  
    errcolor='.26', errwidth=None, capsize=None, dodge=True,  
    ax=None, **kwargs  
)
```

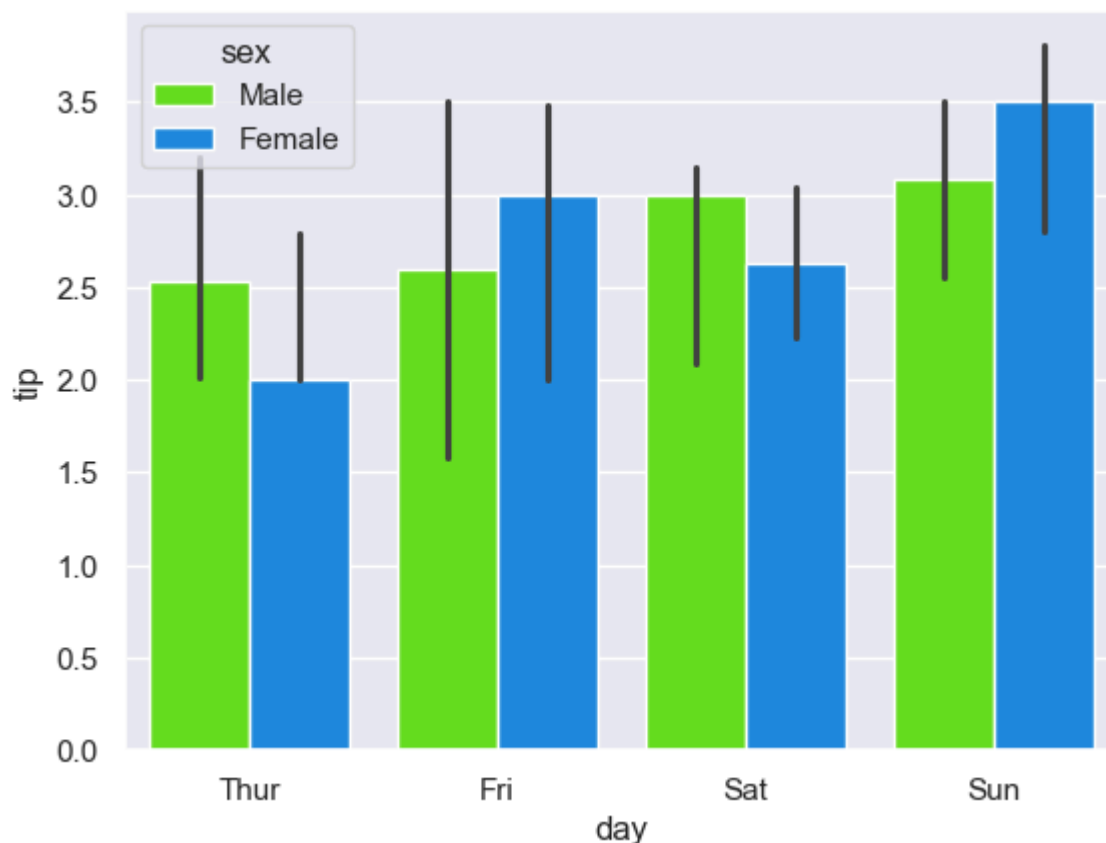
Parameter	Description
<code>x, y</code>	Names of variables in <code>data</code> or vector data. Inputs for plotting long-form data .
<code>hue</code>	(optional) Column name for color encoding .
<code>data</code>	(optional) Pandas DataFrame, array, or list of arrays. If <code>x</code> and <code>y</code> are absent, this is interpreted as wide-form .
<code>estimator</code>	(optional) Statistical function to estimate central tendency (default is <code>mean</code>).
<code>ci</code>	(optional) Size of the confidence interval to draw around estimated values (default is <code>95</code>).
<code>n_boot</code>	(optional) Number of bootstrap iterations used to compute confidence intervals.
<code>color</code>	(optional) Matplotlib color or seed for a gradient palette for all bars.
<code>palette</code>	(optional) Color palette for hue levels.
<code>saturation</code>	(optional) Saturation of colors.
<code>errcolor</code>	(optional) Color of the error bars.
<code>errwidth</code>	(optional) Width of the error bars.
<code>capsize</code>	(optional) Width of the error bar caps.
<code>dodge</code>	(optional) If <code>True</code> , separates bars for different hue levels.
<code>ax</code>	(optional) Axes object to draw the plot onto.
<code>`kwargs`**</code>	Additional keyword arguments passed to Matplotlib's <code>Axes.bar()</code> .

In [103...

```
from numpy import median
sns.barplot(x='day',y='tip',data=df,hue='sex',palette='gist_rainbow',estimator=median)
```

Out[103...

```
<Axes: xlabel='day', ylabel='tip'>
```



Stripplot

What is a Strip Plot?

A **strip plot** is a type of scatter plot where **one variable is categorical**.

It's typically used to **display all observations along a category axis**, making it a good complement to a boxplot or violinplot by showing the actual data points over the distribution.

When to Use

-  To visualize **individual data points across categories**.
 -  Good for **small datasets** or when you want to emphasize individual observations.
 -  Can combine with **boxplot** or **violinplot** for added context.
-

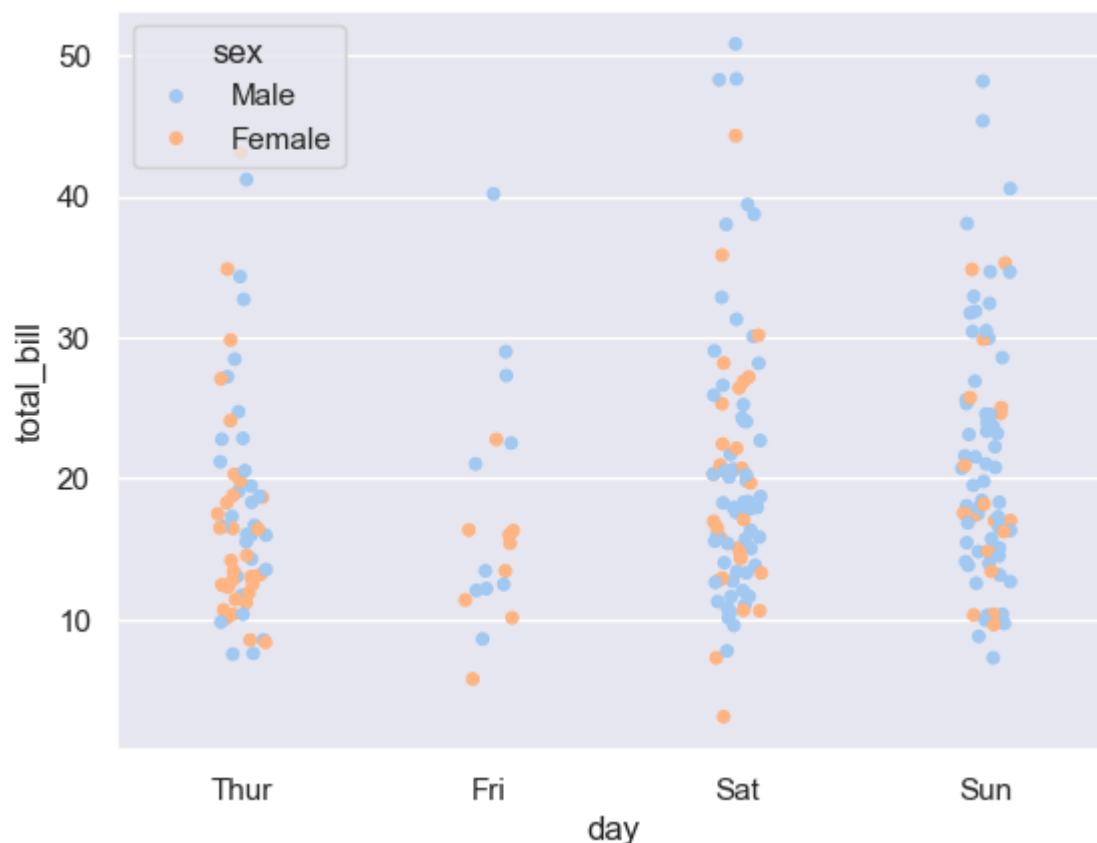
Syntax

```
seaborn.stripplot(  
    *, x=None, y=None, hue=None, data=None, order=None, hue_order=None,  
    jitter=True, dodge=False, orient=None, color=None, palette=None,  
    size=5, edgecolor='gray', linewidth=0, ax=None, **kwargs  
)
```

 Parameter	 Description
<code>x, y, hue</code>	Inputs for plotting long-form data .
<code>data</code>	Dataset for plotting (usually a Pandas DataFrame).
<code>order</code>	Order to plot the categorical levels.
<code>color</code>	Color for all of the elements, or seed for a gradient palette.
<code>palette</code>	(optional) Color palette for hue levels.
<code>jitter</code>	(optional) Add some random noise to spread the points for visibility (default <code>True</code>).
<code>dodge</code>	(optional) When hue nesting is used, separates points by hue value (default <code>False</code>).
<code>size</code>	Size of the points (default <code>5</code>).
<code>edgecolor</code>	Color of point borders (default <code>'gray'</code>).
<code>linewidth</code>	Width of point borders (default <code>0</code>).
<code>ax</code>	Matplotlib Axes to draw the plot onto.
<code>`kwargs`</code>	Additional keyword arguments for Matplotlib's plotting functions.

```
In [106... sns.stripplot(x="day", y="total_bill", hue='sex', data=df, jitter=True, palette="pastel")
```

```
Out[106... <Axes: xlabel='day', ylabel='total_bill'>
```



Swarmplot



What is a Swarm Plot?

A **Swarm Plot** is a categorical scatter plot where the points are **adjusted (or "swarmed") along the categorical axis** so they don't overlap.

It's essentially like a strip plot but positions points to avoid overlap while still showing **all individual observations**.



Why and When to Use Swarm Plot

-  **Displays all individual data points clearly** without overlap, making it superior to a strip plot when the dataset has many overlapping values.
-  Useful for **visualizing distribution patterns, clusters, and outliers** within each category.
-  Can be combined with **boxplot** or **violinplot** to add detailed distribution context while preserving raw data visibility.
-  Best for **small to moderately sized datasets** because with too many points, it may look cluttered.



Syntax

```
seaborn.swarmplot(parameters)
```

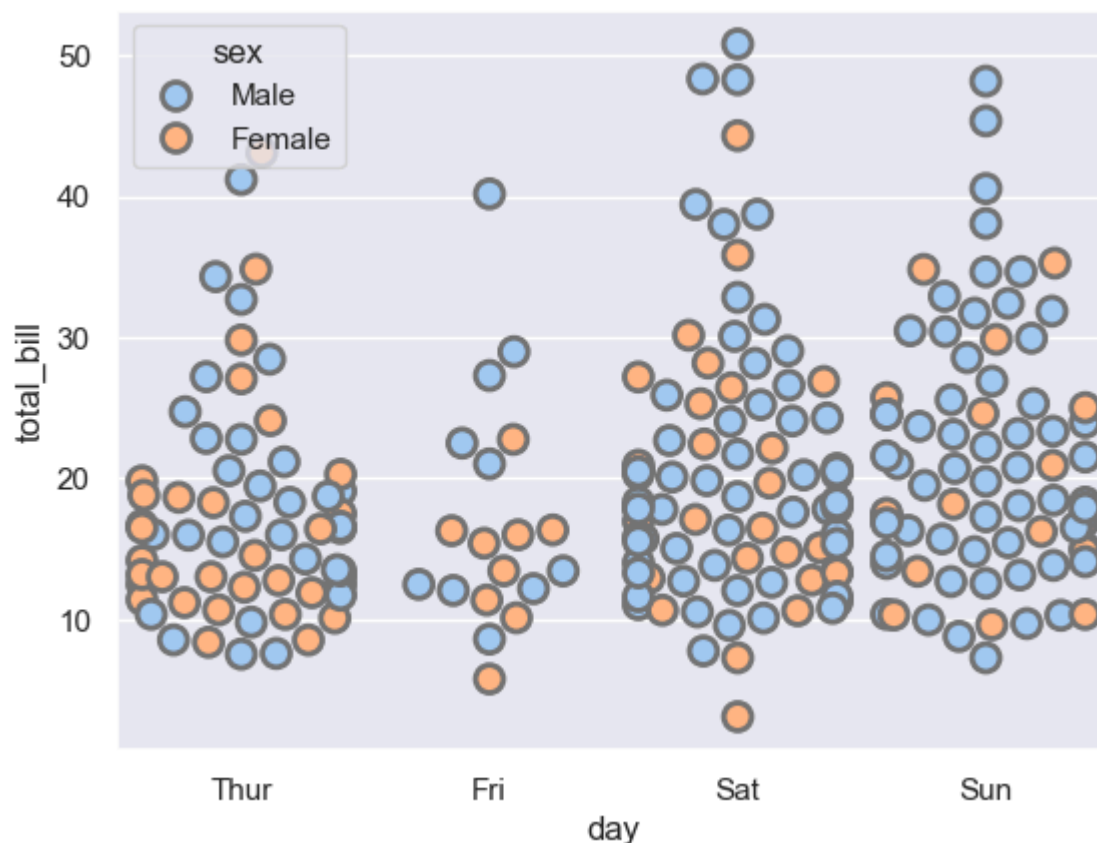
 Parameter	 Description
<code>x</code> , <code>y</code> , <code>hue</code>	Inputs for plotting long-form data (column names or vectors).
<code>data</code>	Dataset for plotting (usually a Pandas DataFrame).
<code>order</code> , <code>hue_order</code>	Order to plot the categorical levels in, otherwise inferred from data.
<code>dodge</code>	Separate the points for different hue levels along the categorical axis.
<code>orient</code>	Orientation of the plot: 'v' (vertical) or 'h' (horizontal).
<code>color</code>	Color for all the points, or seed for a gradient palette.
<code>palette</code>	Colors to use for different hue levels.
<code>size</code>	Radius of the markers (in points).
<code>edgecolor</code>	Color of the lines around each point.
<code>linewidth</code>	Width of the gray lines framing the points.
<code>ax</code>	Axes object to draw the plot onto (optional).
<code>`kwargs`**</code>	Additional keyword arguments for Matplotlib's plotting functions.

In [117... `sns.swarmplot(x="day", y="total_bill", hue='sex', data=df, orient='v', size=10, linewidth=2, pal`

```
c:\Users\ajink\AppData\Local\Programs\Python\Python312\Lib\site-packages\seaborn\categorical.p
y:3399: UserWarning: 21.0% of the points cannot be placed; you may want to decrease the size o
f the markers or use stripplot.
  warnings.warn(msg, UserWarning)
c:\Users\ajink\AppData\Local\Programs\Python\Python312\Lib\site-packages\seaborn\categorical.p
y:3399: UserWarning: 24.1% of the points cannot be placed; you may want to decrease the size o
f the markers or use stripplot.
  warnings.warn(msg, UserWarning)
c:\Users\ajink\AppData\Local\Programs\Python\Python312\Lib\site-packages\seaborn\categorical.p
y:3399: UserWarning: 18.4% of the points cannot be placed; you may want to decrease the size o
f the markers or use stripplot.
  warnings.warn(msg, UserWarning)
```

Out[117... `<Axes: xlabel='day', ylabel='total_bill'>`

```
c:\Users\ajink\AppData\Local\Programs\Python\Python312\Lib\site-packages\seaborn\categorical.p
y:3399: UserWarning: 25.8% of the points cannot be placed; you may want to decrease the size o
f the markers or use stripplot.
  warnings.warn(msg, UserWarning)
c:\Users\ajink\AppData\Local\Programs\Python\Python312\Lib\site-packages\seaborn\categorical.p
y:3399: UserWarning: 32.2% of the points cannot be placed; you may want to decrease the size o
f the markers or use stripplot.
  warnings.warn(msg, UserWarning)
c:\Users\ajink\AppData\Local\Programs\Python\Python312\Lib\site-packages\seaborn\categorical.p
y:3399: UserWarning: 28.9% of the points cannot be placed; you may want to decrease the size o
f the markers or use stripplot.
  warnings.warn(msg, UserWarning)
```



Kdeplot



What is a KDE Plot?

A **Kernel Density Estimate (KDE) Plot** is a method for visualizing the **probability density function (PDF)** of a **continuous or non-parametric variable**.

It represents the distribution of data by plotting a **smoothed curve** derived from the data points.



Why Use a KDE Plot?

-  Helps estimate and visualize the **underlying distribution** of continuous data.
-  Can plot **multiple KDE curves in a single graph** for comparison.
-  Provides a **smooth, continuous visualization** compared to a histogram.
-  Can be extended to visualize **2D KDE plots** for bivariate distributions.



Syntax

```
seaborn.kdeplot( x=None, , y=None, vertical=False, palette=None, *kwargs )
```

 Parameter	 Description
x , y	Vectors or keys in <code>data</code> for plotting the distribution.
vertical	Boolean (<code>True</code> or <code>False</code>). If <code>True</code> , KDE curve is drawn vertically.
data	Pandas DataFrame, NumPy ndarray, mapping, or sequence containing the data.

Parameter

Description

palette	Color palette for multiple KDE curves.
<code>`kwargs`**</code>	Additional keyword arguments passed to Matplotlib plotting functions.

In [124...

df

Out[124...

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4
...	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3
240	27.18	2.00	Female	Yes	Sat	Dinner	2
241	22.67	2.00	Male	Yes	Sat	Dinner	2
242	17.82	1.75	Male	No	Sat	Dinner	2
243	18.78	3.00	Female	No	Thur	Dinner	2

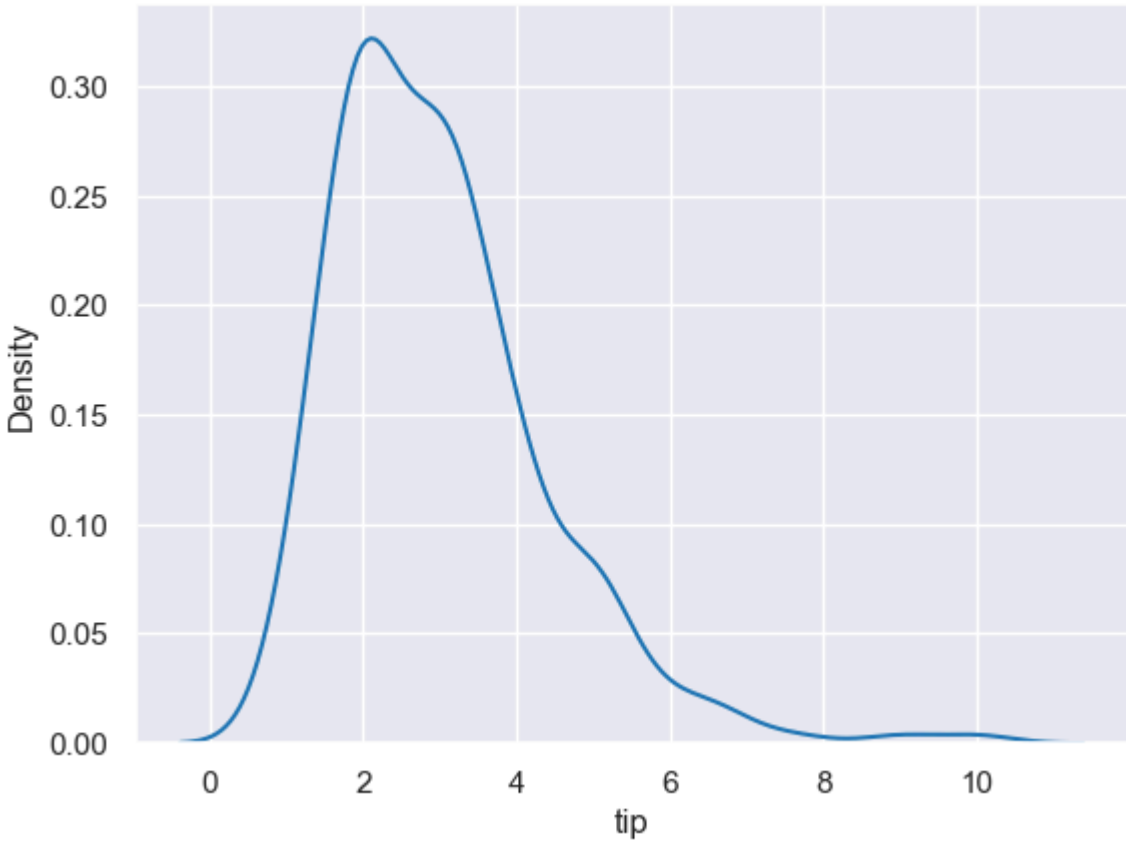
244 rows × 7 columns

In [137...

sns.kdeplot(x='tip',data=df)

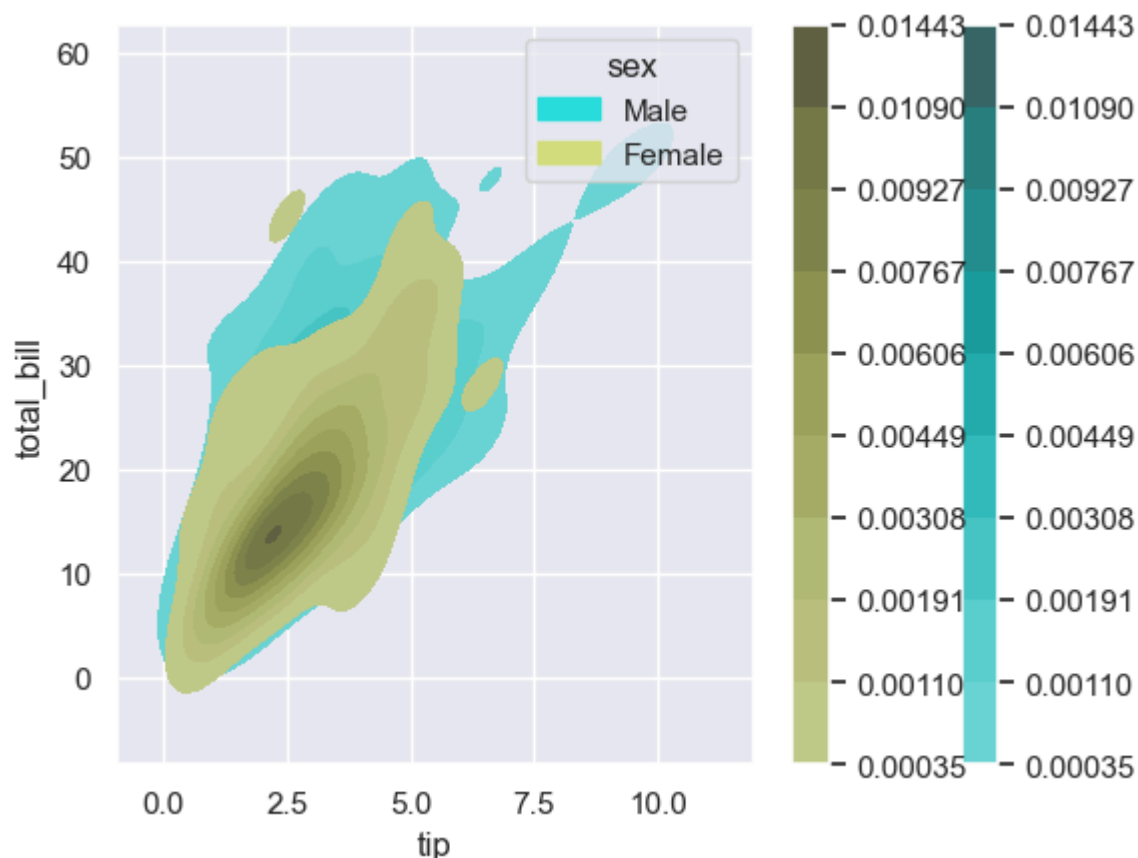
Out[137...

<Axes: xlabel='tip', ylabel='Density'>



```
In [136... sns.kdeplot(x='tip',y='total_bill',data=df,hue='sex',fill=True,palette='rainbow',cbar=True)
```

```
Out[136... <Axes: xlabel='tip', ylabel='total_bill'>
```



What is `catplot()` in Seaborn?

The `catplot()` function in Seaborn is a **figure-level function for drawing categorical plots** onto a `FacetGrid`.

It acts as a **unified interface** to multiple types of categorical plots like **strip, swarm, box, violin, boxen, point, bar, and count** plots.

Syntax

```
seaborn.catplot(  
    *, x=None, y=None, hue=None, data=None, row=None, col=None,  
    kind='strip', color=None, palette=None, **kwargs  
)
```



Parameter



Description

`x`, `y`,
`hue`

Names of variables in `data`. Inputs for plotting **long-form (tidy) data**.

`data`

Pandas DataFrame. Long-form dataset for plotting. Each column should be a variable, each row an observation.

`row`, `col`

Names of categorical variables in `data` for faceting subplots across rows and columns.

`kind`

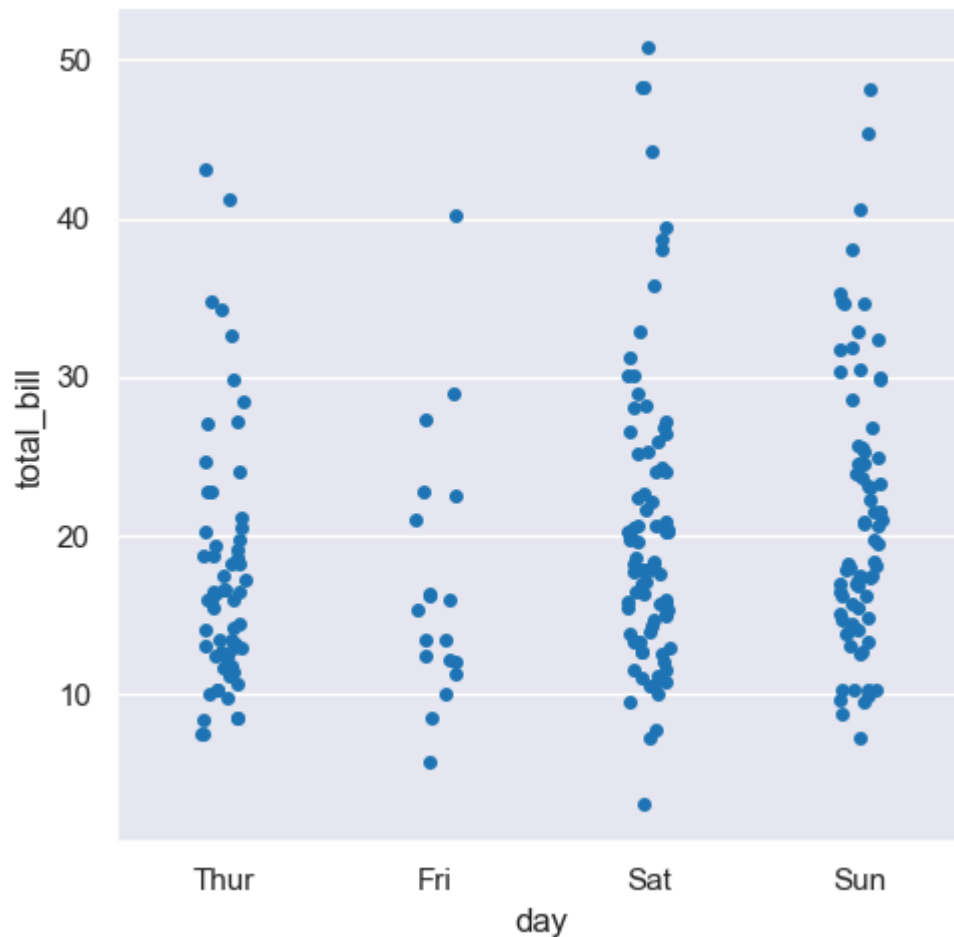
Type of plot to draw. Options: `"strip"`, `"swarm"`, `"box"`, `"violin"`, `"boxen"`, `"point"`, `"bar"`, `"count"`.

 Parameter	 Description
---	--

color	Matplotlib color or seed for a gradient palette (applied to all elements).
palette	Palette name, list, or dictionary mapping hue levels to colors.
`kwargs`**	Additional keyword arguments passed to the underlying plotting function.

```
In [141]: sns.catplot(x="day", y="total_bill", data=df)
```

```
Out[141]: <seaborn.axisgrid.FacetGrid at 0x1feffaf1cd0>
```



```
In [2]: import os
import subprocess
import pdfkit

notebook_file = r"D:\study\LIBRARIES\Seaborn.ipynb"
output_pdf = r"D:\study\LIBRARIES\Seaborn.pdf"
wkhtmltopdf_path = r"C:\Program Files\wkhtmltopdf\bin\wkhtmltopdf.exe" # Update if different

# --- Step 1: Convert Notebook to HTML ---
notebook_dir = os.path.dirname(notebook_file)
notebook_name = os.path.basename(notebook_file)
os.chdir(notebook_dir)

try:
    subprocess.run(["jupyter", "nbconvert", "--to", "html", notebook_name], check=True)
except subprocess.CalledProcessError as e:
    print("Error converting notebook to HTML:", e)
    exit(1)

html_file = os.path.join(notebook_dir, notebook_name.replace(".ipynb", ".html"))
```

```
# --- Step 2: Convert HTML to PDF using pdfkit ---
config = pdfkit.configuration(wkhtmltopdf=wkhtmltopdf_path)
pdfkit.from_file(html_file, output_pdf, configuration=config)

print(f"PDF saved to: {output_pdf}")
```

```
-----
FileNotFoundError                                Traceback (most recent call last)
File c:\Users\ajink\AppData\Local\Programs\Python\Python312\Lib\site-packages\pdfkit\configuration.py:35, in Configuration.__init__(self, wkhtmltopdf, meta_tag_prefix, environ)
    33     self.wkhtmltopdf = lines[0].strip()
--> 35 with open(self.wkhtmltopdf) as f:
    36     pass
```

FileNotFoundError: [Errno 2] No such file or directory: 'C:\\Program Files\\wkhtmltopdf\\bin\\wkhtmltopdf.exe'

During handling of the above exception, another exception occurred:

```
OSError                                           Traceback (most recent call last)
Cell In[2], line 23
    20 html_file = os.path.join(notebook_dir, notebook_name.replace(".ipynb", ".html"))
    22 # --- Step 2: Convert HTML to PDF using pdfkit ---
--> 23 config = pdfkit.configuration(wkhtmltopdf=wkhtmltopdf_path)
    24 pdfkit.from_file(html_file, output_pdf, configuration=config)
    26 print(f"PDF saved to: {output_pdf}")

File c:\Users\ajink\AppData\Local\Programs\Python\Python312\Lib\site-packages\pdfkit\api.py:86, in configuration(**kwargs)
    78 def configuration(**kwargs):
    79     """
    80     Constructs and returns a :class:`Configuration` with given options
    81
    82     :param wkhtmltopdf: path to binary
    83     :param meta_tag_prefix: the prefix for ``pdfkit`` specific meta tags
    84     """
--> 86     return Configuration(**kwargs)

File c:\Users\ajink\AppData\Local\Programs\Python\Python312\Lib\site-packages\pdfkit\configuration.py:38, in Configuration.__init__(self, wkhtmltopdf, meta_tag_prefix, environ)
    36     pass
    37 except (IOError, FileNotFoundError) as e:
--> 38     raise IOError('No wkhtmltopdf executable found: "%s"\n'
    39                    'If this file exists please check that this process can '
    40                    'read it or you can pass path to it manually in method call, '
    41                    'check README. Otherwise please install wkhtmltopdf - '
    42                    'https://github.com/JazzCore/python-pdfkit/wiki/Installing-wkhtmltopdf' % self.wkhtmltopdf)
    45 self.environ = environ
    47 if not self.environ:
```

OSError: No wkhtmltopdf executable found: "C:\Program Files\wkhtmltopdf\bin\wkhtmltopdf.exe"
 If this file exists please check that this process can read it or you can pass path to it manually in method call, check README. Otherwise please install wkhtmltopdf - <https://github.com/JazzCore/python-pdfkit/wiki/Installing-wkhtmltopdf>